Temporizadores de hardware en el ESP32

Sistemas Digitales · 5.º ciclo de Computación · UNL

Unidad 1: Microcontroladores para sistemas embebidos

Semana 3 · Miércoles 28 de octubre de 2026

Objetivo de la sesión

Al terminar, podrás explicar cómo un temporizador mide el tiempo por hardware, calcular el valor de alarma para un tiempo dado y describir cómo el temporizador avisa a la CPU.

1 **Explicar**

qué es un temporizador y cómo cuenta

2 **Calcular**

resolución y valor de alarma

3 **Describir**

la interrupción y cómo la CPU se entera

Resultado de aprendizaje R1: diseña arquitectura de microcontroladores.

Repaso de la semana 2

El número de GPIO es un nombre, no una posición

30

pines en la placa DevKit V1

25

de esos pines son GPIO

34

GPIO existen en el chip ESP32

- GPIO18 es el nombre de una señal del chip, no "el pin número 18 de la placa".
- En el chip no existen GPIO20, GPIO24 ni GPIO28 a GPIO31: la numeración tiene huecos.
- Siempre se busca el rótulo impreso en la placa y el diagrama de pines.

Los GPIO 34 a 39 solo sirven como entrada

GPIO	Rótulo frecuente	¿Salida?	¿Pull-up interna?
34, 35	D34, D35	No	No
36	VP	No	No
39	VN	No	No

Por qué

No tienen driver de salida: el circuito que entrega corriente hacia afuera. Tampoco tienen resistencia de pull-up interna. Solo "escuchan".

Un LED en el GPIO34 nunca se enciende, aunque el código compile sin errores.

Variable local frente a variable static

Local = vaso desechable

Se crea al entrar a la función y se tira al salir. En cada vuelta de loop() empieza de nuevo con su valor inicial.

static = taza en tu escritorio

Se crea una sola vez y se queda. Cada vuelta de loop() encuentra el valor que dejó la anterior.

```
void loop() {
  int    a = 0;  a++;   // a vale 1 en TODAS las vueltas
  static int b = 0; b++; // b vale 1, 2, 3, 4...
}
```

Promoción a int: el compilador convierte antes de operar

```
uint8_t x = 200, y = 100;
uint8_t r = x + y;   // r = 44, no 300

if (-1 < 1u) { ... } // FALSO
```

Regla

Antes de operar, los tipos pequeños (uint8_t, char) se convierten a int de 32 bits. El resultado se recorta al guardarlo.

- x + y se calcula como int: 300. Al guardarlo en 8 bits queda 300 − 256 = 44.
- -1 < 1u: al mezclar con un unsigned, -1 se convierte a 4 294 967 295, que no es menor que 1.
- Lección: en embebidos, el tamaño real de cada tipo importa.

¿Qué es un temporizador?

Qué es, cómo cuenta y quién lo controla

Temporizador (timer): definición

Circuito dentro del microcontrolador que cuenta pulsos de reloj. Como cada pulso dura un tiempo conocido, sirve para medir tiempo y para avisar cuando ha pasado un tiempo determinado.

Temporizador = timer

Es la misma cosa: una palabra en español y otra en inglés.

Es hardware

No es un programa: es un circuito fabricado dentro del chip, como los pines GPIO.

Es un periférico

Bloque de hardware del chip con una función específica, aparte de la CPU.

¿Por qué no basta con delay() o millis()?

`delay(1000)`

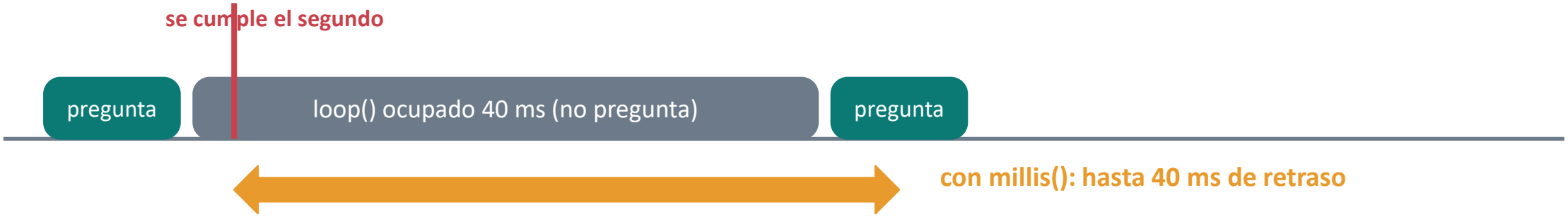
La CPU se queda esperando sin hacer nada más.

`millis()`

La CPU pregunta "¿ya pasó?" en cada vuelta de loop().

Temporizador

Un circuito aparte cuenta y avisa a la hora exacta.



El contador es un registro del temporizador

Registro

Espacio pequeño de almacenamiento de bits. Los periféricos tienen los suyos, con dirección en el mapa de memoria (semana 1).

Contador

Registro del temporizador con un circuito sumador pegado: le suma 1 en cada pulso, sin que nadie se lo pida.

Diferencia con la RAM: una celda de RAM solo guarda lo que le escriben. El contador cambia solo. No está en la RAM de las variables, sino dentro del circuito del temporizador.

¿Cómo cuenta? En binario, empezando normalmente en 0

```
0000 ... 0000   →  0
0000 ... 0001   →  1
0000 ... 0010   →  2
0000 ... 0011   →  3
0000 ... 0100   →  4
```

en el registro (binario) como lo escribimos

- Internamente suma en binario: el hardware solo maneja bits.
- Nosotros lo escribimos en decimal por comodidad; es el mismo número.
- Al configurar el temporizador, el contador queda en 0.
- No suma en cada pulso del reloj, sino en cada pulso que sale del prescaler (lo vemos pronto).

¿Quién lo arranca y quién lo detiene? La CPU

1

La CPU configura

Escribe los valores en los registros del temporizador.

2

La CPU lo habilita

Escribe 1 en el bit de habilitación (enable).

3

El temporizador cuenta solo

La CPU queda libre para otras tareas.

4

La CPU lo detiene

Escribe 0 en el bit: el contador se congela.

! **La CPU no ejecuta ninguna instrucción por cada conteo. Solo configura, arranca y detiene.**

El reloj, el prescaler y las fórmulas

Cómo se convierte un conteo en tiempo

El reloj del temporizador: 80 millones de pulsos por segundo

1 **Reloj**

Señal que sube y baja a ritmo fijo, como un metrónomo. Cada subida es un pulso.

2 **APB**

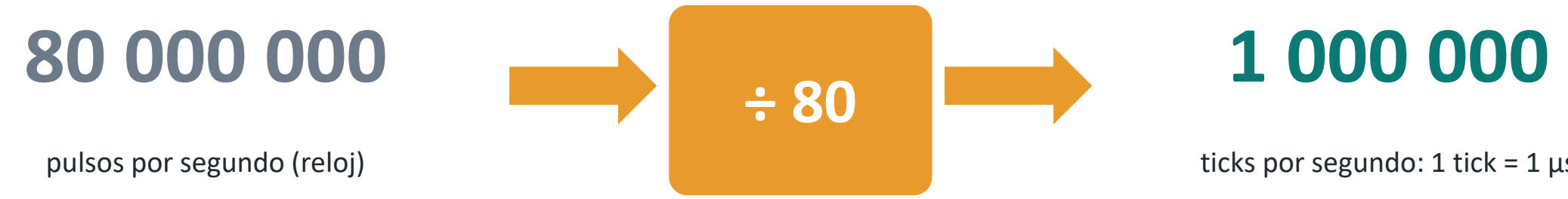
Bus de periféricos del ESP32. Su reloj, de 80 MHz, es el que llega a los temporizadores.

3 **80 MHz**

80 000 000 pulsos por segundo: cada pulso dura 12,5 nanosegundos.

! **Si el contador sumara con cada pulso, para medir 1 segundo tendría que llegar a 80 000 000.**

El prescaler: un divisor antes del contador



Prescaler: divisor que deja pasar al contador solo 1 de cada N pulsos del reloj. El contador avanza N veces más lento.

Tick del timer: cada pulso que sale del prescaler; es cuando el contador suma 1.

¿Para qué sirve el prescaler y cuándo usarlo?

1. Elegir una unidad cómoda

Con N = 80, cada tick vale 1 µs: el contador "cuenta en microsegundos". Es el uso principal en el ESP32.

2. Medir tiempos largos

Un contador pequeño se llena rápido. Con ticks más lentos tarda más en llenarse. Es clave en el AVR.

N pequeño

ticks cortos: más precisión, el contador se llena antes



N grande

ticks largos: cubre más tiempo, menos precisión

Regla para la clase: ticks de 1 µs (1 MHz).

Resolución: lo que dura un tick

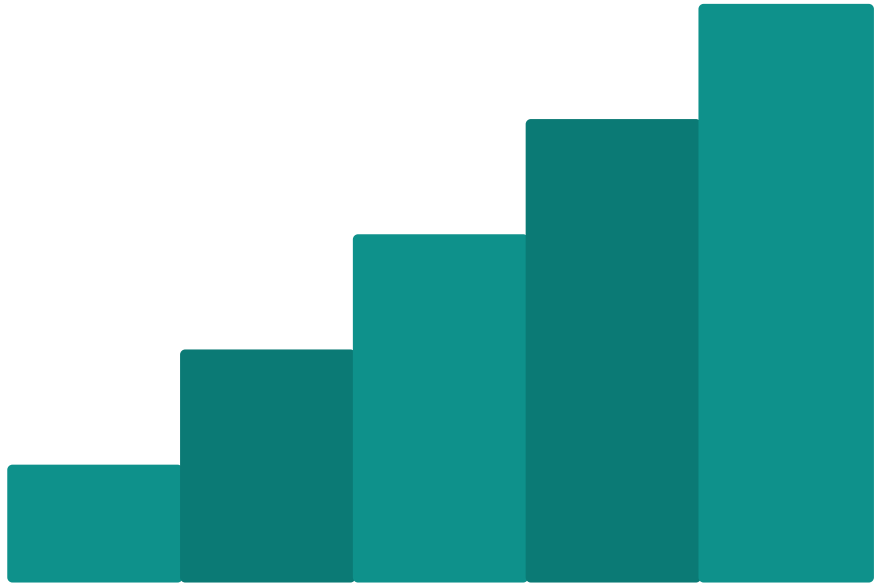
Resolución: duración de un tick. Es el tiempo más pequeño que el temporizador puede distinguir.

$$\text{resolución} = 1 / f$$

f = ticks por segundo (la frecuencia que sale del prescaler)

f (ticks por segundo)	Resolución
1 000 000 (1 MHz)	1 µs
10 000 (10 kHz)	100 µs

La alarma y la fórmula del tiempo: la escalera



altura de un peldaño = resolución

número de peldaños = alarma

Alarma: número que se guarda en un registro. Cuando el contador lo alcanza, el temporizador genera un aviso.

tiempo = alarma × resolución

despejando: alarma = tiempo × f

Con f = 1 MHz, la alarma se escribe directamente en microsegundos.

Ejemplo resuelto 1: un aviso cada medio segundo

Datos: f = 1 000 000 ticks por segundo · tiempo deseado = 0,5 s

1. **Resolución** — $1 / 1\,000\,000\ \text{s} = 1\ \mu\text{s}$
2. **Alarma** — $0{,}5\ \text{s} \times 1\,000\,000 = 500\,000$
3. **Comprobación** — $500\,000 \times 1\ \mu\text{s} = 0{,}5\ \text{s}$
4. **Prescaler** — $N = 80\,000\,000 / 1\,000\,000 = 80$

Ejemplo resuelto 2: otra frecuencia de tick

Datos: f = 10 000 ticks por segundo · alarma = 2500

1 **Resolución** $1 / 10\,000\ \text{s} = 100\ \mu\text{s}$

2 **Tiempo** $2500 \times 100\ \mu\text{s} = 0{,}25\ \text{s}$ (4 avisos por segundo)

3 **Prescaler** $N = 80\,000\,000 / 10\,000 = 8000$ (válido)

! **Mismo método, otros números: peldaño de 100 µs, 2500 peldaños.**

Actividad en parejas: tres cálculos rápidos

A

f = 1 000 000 y alarma = 2000.

¿Cada cuánto ocurre el aviso?

B

Quiero un aviso cada 10 ms con f = 1 000 000.

¿Qué alarma uso?

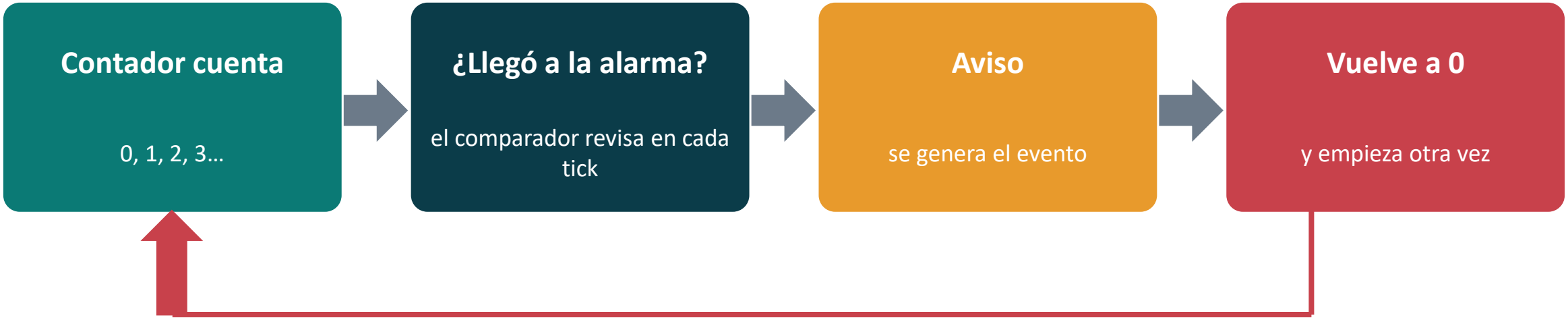
C

f = 40 000 y alarma = 40 000.

¿Cada cuánto ocurre el aviso? ¿Qué prescaler N se necesita?

5 minutos. Escriban la fórmula usada, no solo el número.

Autorrecarga: el aviso se repite solo



Autorrecarga (auto-reload): al alcanzar la alarma, el contador vuelve solo a 0. Así el aviso se repite a intervalos iguales sin que el programa haga nada.

Desbordamiento no es lo mismo que autorrecarga

	Alarma con autorrecarga	Desbordamiento
Qué valor lo provoca	Uno que tú eliges (ej.: 500 000)	El máximo que cabe en los bits
Qué pasa	Aviso y el contador vuelve a 0	El contador "da la vuelta" a 0
¿Es intencional?	Sí: es el uso normal	Casi nunca: es el límite físico

Pista de atletismo

La alarma es la línea que tú pintas en el metro 400. El desbordamiento es el final de la pista: si no hubiera línea, el corredor llegaría hasta ahí y empezaría de nuevo por obligación.

¿Cuál es el límite del contador?

Con n bits, el máximo es $2^n - 1$

Contador	Valor máximo	Con ticks de 1 µs, se llena en…
8 bits (Timer0 del AVR)	255	0,26 ms
16 bits (Timer1 del AVR)	65 535	65,5 ms
64 bits (ESP32)	18 446 744 073 709 551 615	≈ 584 000 años

! **En el ESP32 el desbordamiento prácticamente no ocurre: se trabaja siempre con la alarma.**

Los temporizadores del ESP32

Cuántos hay y en qué se diferencian del AVR

El ESP32 tiene 4 temporizadores de propósito general

Grupo TIMG0

Timer 0

contador de 64 bits
prescaler de 16 bits

Timer 1

contador de 64 bits
prescaler de 16 bits

Grupo TIMG1

Timer 0

contador de 64 bits
prescaler de 16 bits

Timer 1

contador de 64 bits
prescaler de 16 bits

- Cada uno cuenta hacia arriba o hacia abajo y tiene autorrecarga.
- Funcionan en paralelo: hasta 4 avisos periódicos independientes.

ESP32 frente al AVR ATmega328P de la semana 1

Característica	ESP32 (WROOM-32)	ATmega328P (Uno)
Temporizadores de uso general	4	3 (Timer0, Timer1, Timer2)
Tamaño del contador	64 bits	8 o 16 bits
Reloj de entrada	80 MHz	16 MHz
Prescaler	Cualquier N de 2 a 65 535	Solo 1, 8, 64, 256 o 1024
Se llena (sin prescaler)	Prácticamente nunca	Timer1: en 4,1 ms

¿Cómo le damos órdenes al temporizador?

Qué es una API, qué es un core y qué cambió en la versión 3

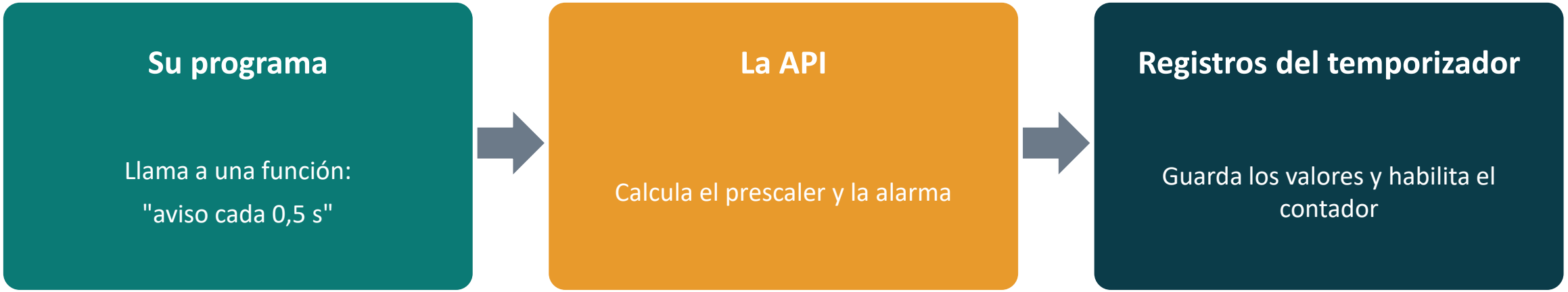
API: el menú del restaurante

API (Application Programming Interface): conjunto de funciones que alguien ya programó para que uno las use sin saber cómo están hechas por dentro.

Menú	Pedido	Cocina
La API: la lista de lo que puedes pedir.	Llamar a una función.	El hardware y los registros: no necesitas entrar.

Ya la han usado: pinMode(), digitalWrite(), delay() y millis() son funciones de una API.

La API escribe los registros por nosotros



- Es lo mismo que hace digitalWrite() con los registros del puerto (semana 1).
- Nosotros no escribiremos los registros del temporizador a mano, pero deben saber que existen y qué guardan.

Core: el paquete que enseña al Arduino IDE a programar el ESP32

Core (núcleo): paquete de software que se instala en el Arduino IDE para programar una placa concreta. Trae el compilador y la API de esa placa.

De fábrica

El Arduino IDE trae el core de las placas Arduino (Uno, Mega). No sabe programar el ESP32.

Para el ESP32

Se instala el core de Espressif, fabricante del chip: "esp32 by Espressif Systems", en Herramientas → Placa → Gestor de placas.

Versión 3.x: cambió la API de los temporizadores

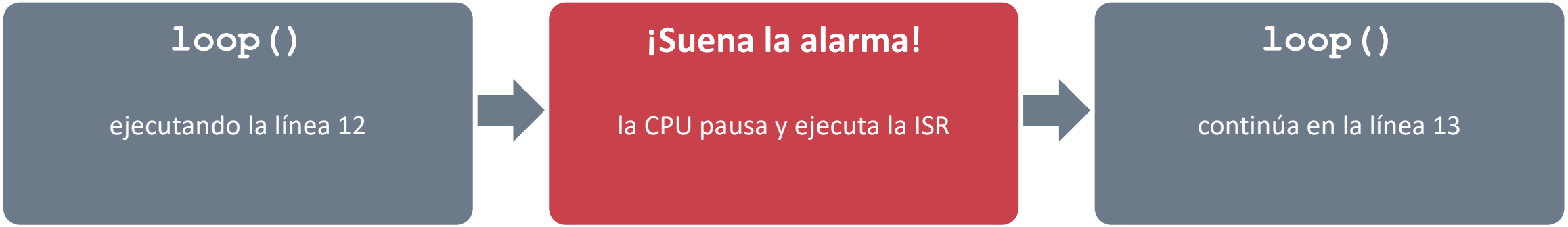
1. **3.x** — Cualquier versión que empiece por 3: 3.0, 3.1, 3.2…
2. **Qué cambió** — Los nombres y parámetros de las funciones de los temporizadores.
3. **Consecuencia** — Un código escrito para la versión 2 no compila en la versión 3.

! **La mayoría de tutoriales de internet usan la versión 2. Copiarlos produce errores de compilación.**

¿Cómo se entera la CPU de que sonó la alarma?

Interrupción, callback, IRAM_ATTR, volatile y patrón bandera. Las interrupciones externas son de la semana 4.

Interrupción: el timbre que te hace dejar lo que haces



Interrupción: señal que un periférico envía a la CPU para decirle "detente, pasó algo". La CPU ejecuta una función especial y vuelve exactamente adonde estaba.

ISR (Interrupt Service Routine): rutina de servicio de interrupción; la función que se ejecuta cuando llega la interrupción.

Interrupción del temporizador (hoy) frente a externa (semana 4)

	Del temporizador (semana 3)	Externa (semana 4)
Quién la dispara	Un periférico interno: el temporizador llega a la alarma	Un pin que cambia de nivel por algo de afuera
Cuándo ocurre	Cada cierto tiempo, de forma predecible	Cuando pasa algo, sin aviso
Ejemplo	Encender un LED cada 2 segundos	Detectar que alguien presionó un botón

! **Lo que aprendemos hoy (ISR, callback, IRAM_ATTR, volatile, bandera) sirve para las dos.**

Callback: dejar tu número para que te llamen

Callback: función que tú escribes, pero que no llamas tú: se la entregas a otro para que él la llame cuando ocurra algo.

1. **Escribes la función** con lo que quieres que pase cuando suene la alarma.
2. **Se la entregas al temporizador** con una función de la API.
3. **El hardware la llama** cada vez que el contador llega a la alarma. Tu programa nunca la llama.

Taller mecánico: "llámeme cuando el auto esté listo". Su número es el callback.

IRAM_ATTR: guardar la ISR donde siempre esté a la mano

Memoria flash

Chip aparte donde se guarda el programa. Grande y no se borra al apagar, pero lenta y a veces ocupada.

IRAM

RAM interna rápida para instrucciones (Instruction RAM), dentro del ESP32. Siempre disponible.

IRAM_ATTR: etiqueta que se escribe delante del nombre de la ISR. Le pide al compilador copiarla a la IRAM.

El extintor no se guarda en el sótano con llave: se cuelga en la pared, a la mano.

volatile: leer siempre el valor real, no una copia

Compilador: traduce el código a instrucciones del chip y además toma atajos para que el programa sea más rápido (optimiza).

Sin volatile

El compilador no ve la ISR. Cree que la variable no cambia, guarda una copia y la reutiliza. loop() nunca ve el cambio.

Con volatile

El compilador sabe que algo externo puede cambiarla. La lee de la memoria cada vez.

! **Regla: toda variable que comparten la ISR y loop() se declara volatile.**

Patrón bandera: la ISR avisa, loop() trabaja

Problema: la ISR debe ser muy corta, pero a veces queremos hacer algo lento cuando suena la alarma, como imprimir un mensaje.

1. La ISR solo avisa

Pone una variable en verdadero (true). Esa variable es la bandera. Termina enseguida.

2. loop() hace el trabajo

En cada vuelta revisa la bandera. Si está levantada, la baja (false) y hace el trabajo lento.

Buzón rural: el cartero deja la carta y levanta la banderita; el dueño la ve, la baja y lee con calma.